Patterns of Distributed Systems

A Practitioner's Guide

Based on Unmesh Joshi's book • All 29 patterns with code examples

Introduction

Distributed systems are hard. Processes crash, networks partition, clocks drift, and latency is unpredictable. Despite this, the world's most critical infrastructure — Kafka, ZooKeeper, etcd, Cassandra, CockroachDB, Google Spanner — runs reliably because the engineers who built them independently discovered the same set of solutions. Unmesh Joshi catalogued those recurring solutions into 29 patterns.

This guide covers all 29 patterns grouped by domain. For each pattern you'll find: what problem it solves, how the solution works, its key advantages, when you absolutely must use it, and working code to make the ideas concrete.

Pattern Quick-Reference

Pattern	Category	Use When
Write-Ahead Log (WAL)	Durability	ANY system that stores state and must survive crashes — databases (Postgres WAL), message brokers (Kafka), consensus systems (etcd, ZooKeeper)
Segmented Log	Durability	Kafka topics, Raft log implementations, database WALs — any high-volume system where the log would otherwise grow unboundedly
Low-Water Mark	Durability	Always paired with WAL and Segmented Log in any replicated system — Kafka consumer offsets, Raft snapshot indexes, database replication slots
Leader and Followers	Replication	Any replicated system needing strong consistency: Kafka partition leaders, etcd, CockroachDB range leaseholders, Redis Sentinel
High-Water Mark	Replication	All leader-follower systems: Raft's commit index, Kafka's high-water mark on each partition, any database replication with read-your-writes guarantee
Replicated Log	Replication	The core of any strongly-consistent distributed system: etcd, ZooKeeper, CockroachDB, YugabyteDB, any Raft/Paxos implementation

Follower Reads	Replication	Read-heavy workloads: MySQL replicas, MongoDB secondaries, Kafka consumers, CassandraDB (tunable consistency), DynamoDB global tables
Majority Quorum	Consensus	Any time you need a system to remain consistent across node failures: leader election, committing log entries, distributed locks
Generation Clock	Consensus	Raft (term), Paxos (ballot), ZooKeeper (epoch), Kafka (controller epoch)
Paxos	Consensus	Google Chubby, Apache Zookeeper's Zab, Google Spanner's TrueTime-based variant
HeartBeat	Failure Detection	Every distributed system uses this: ZooKeeper session timeouts, Kafka broker liveness, Kubernetes pod health checks, TCP keepalive, Raft election timeouts
Single-Socket Channel	Communication	ZooKeeper client-server communication, Raft inter-node RPCs, Kafka broker connections
Request Pipeline	Communication	High-throughput log append scenarios, Kafka producer acks, Redis pipelining, any network-bound replication path
Request Batch	Communication	Kafka producer batching, Cassandra batch writes, gRPC streaming, database bulk inserts, any high-throughput producer-consumer system
Request Waiting List	Communication	Raft leaders waiting for follower acks before responding to clients, ZooKeeper quorum writes, any leader that must collect peer votes before replying
Singular Update Queue	Communication	ZooKeeper's request processor pipeline, Kafka's log append path, actor frameworks (Akka), any component whose internal state must be updated serially
Fixed Partitions	Partitioning	Kafka (fixed partition count per topic), Redis Cluster (16,384 hash slots), Cassandra (virtual nodes)
Key-Range Partitions	Partitioning	HBase, Cassandra (compound key ranges), CockroachDB, BigTable, DynamoDB table scans, time-series databases
Two-Phase Commit (2PC)	Partitioning	Distributed databases (MySQL NDB, CockroachDB), XA transactions, any operation that must atomically affect data on multiple independent nodes
Versioned Value	Distributed Time	etcd (MVCC store), CockroachDB (MVCC), Cassandra (timestamps), git (commit hashes)
Version Vector	Distributed Time	Multi-master replication (Amazon Dynamo, Riak, CouchDB), shopping cart merges, any leaderless eventual-consistency system
Lamport Clock	Distributed Time	Distributed databases needing causal ordering, distributed debugging/tracing, event sourcing across services
Hybrid Clock	Distributed Time	CockroachDB, YugabyteDB, MongoDB oplog timestamps
Clock-Bound Wait	Distributed Time	Google Spanner, CockroachDB's commit-wait, any globally distributed database that promises linearizable transactions across data centers
Consistent Core	Cluster Management	Kafka + ZooKeeper/KRaft, HDFS + ZooKeeper, Kubernetes + etcd, Elasticsearch + ZooKeeper

Lease	Cluster Management	Kubernetes leader election, GFS master leases, distributed locks (etcd lease TTL), HBase region server assignment, any system needing fencing tokens
State Watch	Cluster Management	ZooKeeper watches (the foundational use case), etcd watch API, Consul health check callbacks, Kubernetes informer/watch pattern
Gossip Dissemination	Cluster Management	Cassandra (failure detection, ring membership), DynamoDB, Riak, Consul, SWIM protocol
Emergent Leader	Cluster Management	Kafka KRaft controller (oldest active broker wins), ZooKeeper leader selection among candidate nodes, Kubernetes lease-based leader election fallback
Idempotent Receiver	Client Patterns	Payment APIs, Kafka producer idempotence (producer ID + sequence number), database upserts with unique keys, any at-least-once messaging system that needs exactly-once semantics

Write-Ahead Log (WAL)

Durability

PROBLEM	When a server crashes mid-operation, in-memory state is lost. How do you guarantee every write survives a crash?

SOLUTION	Before changing any data structure, append an entry describing the intended change to a durable, append-only log file. On restart, replay the log to reconstruct state.

Advantages

- Crash recovery: replay the log to restore exact state
- Sequential writes are far faster than random writes
- Replication: send log entries to replicas instead of full state

USE THIS WHEN	ANY system that stores state and must survive crashes — databases (Postgres WAL), message brokers (Kafka), consensus systems (etcd, ZooKeeper).

Code Example

```
// Simplified WAL in Java
class WriteAheadLog {
  private final Path logPath;
  private final List<LogEntry> log = new ArrayList<>();

  // Every state change MUST go here first
  public long append(byte[] data) throws IOException {
    long entryId = log.size();
    LogEntry entry = new LogEntry(entryId, data, System.currentTimeMillis());
    // Append to disk BEFORE applying to memory
    Files.write(logPath, entry.serialize(),
        StandardOpenOption.APPEND, StandardOpenOption.SYNC);
    log.add(entry);
    return entryId;
  }

  // On startup, call this before accepting any requests
  public void recover(StateMachine sm) throws IOException {
    for (LogEntry e : readAllFromDisk()) {
      sm.apply(e.getData());
    }
  }
}
```

Segmented Log

Durability

PROBLEM	A single, ever-growing WAL file becomes slow to read, hard to truncate, and risky (one corrupt byte loses everything).

SOLUTION	Split the log into fixed-size segment files (e.g., 1 GB each). The active segment is written to; old segments are read-only and can be independently deleted or archived.

Advantages

- Old segments can be deleted when all replicas have caught up (via Low-Water Mark)
- Parallel reads across segments for faster recovery
- Easier to archive cold data (ship old segments to object storage)

USE THIS WHEN	Kafka topics, Raft log implementations, database WALs — any high-volume system where the log would otherwise grow unboundedly.

Code Example

```
class SegmentedLog {
  private final long maxSegmentSize;
  private final List<LogSegment> segments = new ArrayList<>();
  private LogSegment activeSegment;

  public void append(LogEntry entry) throws IOException {
    if (activeSegment.size() >= maxSegmentSize) {
      activeSegment.close();          // seal current segment
      activeSegment = createNewSegment();
    }
    activeSegment.append(entry);
  }

  // Remove segments whose highest offset < lowWatermark
  public void truncateBefore(long lowWatermark) {
    segments.removeIf(s -> s.getLastOffset() < lowWatermark);
  }
}
```

Low-Water Mark

Durability

PROBLEM	Once a WAL (or Segmented Log) is in use, how do you know which old log entries are safe to delete?

SOLUTION	Track the lowest log index that any live consumer (follower, subscriber) still needs. Entries below this index can be safely discarded.

Advantages

- Prevents unbounded log growth
- Coordinates safely across consumers — no data deleted prematurely
- Simple: just track a single monotone index

USE THIS WHEN	Always paired with WAL and Segmented Log in any replicated system — Kafka consumer offsets, Raft snapshot indexes, database replication slots.

Code Example

```
class ReplicationLog {
  // Each follower reports its confirmed index
  private final Map<String, Long> followerIndexes = new ConcurrentHashMap<>();

  public void updateFollowerIndex(String nodeId, long index) {
    followerIndexes.put(nodeId, index);
  }

  // Safe to delete anything below this value
  public long getLowWaterMark() {
    return followerIndexes.values().stream()
        .min(Long::compare)
        .orElse(0L);
  }

  public void garbageCollect(SegmentedLog log) {
    log.truncateBefore(getLowWaterMark());
  }
}
```

Leader and Followers

Replication

PROBLEM	Multiple servers hold copies of the same data. If clients can write to any server, conflicting updates will make replicas diverge.

SOLUTION	Elect exactly one server as the Leader for each data set. All writes go to the leader; the leader propagates changes to Followers. Reads can go to the leader or (with caveats) to followers.

Advantages

- Serializes writes — eliminates write-write conflicts
- Simple, well-understood model (used by Raft, ZooKeeper, Kafka)
- Leader knows the authoritative state at any point

USE THIS WHEN	Any replicated system needing strong consistency: Kafka partition leaders, etcd, CockroachDB range leaseholders, Redis Sentinel.

Code Example

```
class Node {
  enum Role { LEADER, FOLLOWER, CANDIDATE }
  private Role role = Role.FOLLOWER;
  private String leaderId;

  public WriteResponse write(WriteRequest req) {
    if (role != Role.LEADER) {
      // Forward to leader, never process locally
      return forwardToLeader(leaderId, req);
    }
    // Leader: append to WAL, then replicate
    long index = wal.append(req.getData());
    replicateToFollowers(index, req.getData());
    return WriteResponse.success(index);
  }
}
```

High-Water Mark

Replication

PROBLEM	The leader has written entry N to its WAL, but not all followers have it yet. If a client reads entry N from a follower, they see data that might be rolled back.

SOLUTION	Track the highest log index that has been replicated to a majority of nodes. Only expose entries up to this index to clients.

Advantages

- Clients never read uncommitted (possibly rolled-back) data

- Simple integer — cheap to track and compare
- Decouples write availability from read consistency

USE THIS WHEN	All leader-follower systems: Raft's commit index, Kafka's high-water mark on each partition, any database replication with read-your-writes guarantee.

Code Example

```
class ReplicatedLog {
  private long highWaterMark = 0;
  private final Map<String, Long> followerAcks = new ConcurrentHashMap<>();

  public void onFollowerAck(String nodeId, long index) {
    followerAcks.put(nodeId, index);
    updateHighWaterMark();
  }

  private void updateHighWaterMark() {
    // Majority = (n/2)+1 nodes (including leader)
    int majority = (followerAcks.size() + 1) / 2 + 1;
    long newHWM = followerAcks.values().stream()
        .sorted(Comparator.reverseOrder())
        .skip(majority - 1)
        .findFirst().orElse(0L);
    highWaterMark = Math.max(highWaterMark, newHWM);
  }

  // Clients may ONLY read up to this index
  public long getHighWaterMark() { return highWaterMark; }
}
```

Replicated Log

Replication

PROBLEM	How do you keep the state of multiple nodes in sync even when network partitions and node crashes occur?

SOLUTION	Use the Write-Ahead Log as the single source of truth and replicate it to all nodes. Every state change is first appended to the log, then sent to followers. Each follower applies log entries in the same order, guaranteeing identical final state.

Advantages

- Order is deterministic: same log entries → same state on every node
- Recovery is just log replay — no complex state transfer needed

- Foundation for both Raft and Paxos

USE THIS WHEN	The core of any strongly-consistent distributed system: etcd, ZooKeeper, CockroachDB, YugabyteDB, any Raft/Paxos implementation.

Code Example

```
// Raft-style replicated log (simplified)
class RaftNode {
  private final WriteAheadLog log;
  private final List<RaftNode> peers;
  private long commitIndex = 0;

  // Leader only
  public long propose(byte[] command) {
    long index = log.append(command);    // Step 1: local WAL
    int acks = sendAppendEntries(index, command); // Step 2: replicate
    if (acks >= majority()) {
      commitIndex = index;              // Step 3: commit
      applyToStateMachine(command);
    }
    return commitIndex;
  }

  // Follower only
  public void onAppendEntries(long index, byte[] command) {
    log.append(command);
    sendAck(index);
  }
}
```

Follower Reads

Replication

PROBLEM	All reads going through the leader create a bottleneck. The leader is already handling all writes; can followers help with reads?

SOLUTION	Route read-only queries to followers. Accept the trade-off that followers may lag slightly behind (eventual consistency for reads). For stricter consistency, the follower first confirms the leader's current commit index before responding.

Advantages

- Horizontally scales read throughput by adding followers
- Reduces latency by routing reads to the geographically closest replica
- Leader is freed to focus on writes

USE THIS WHEN	Read-heavy workloads: MySQL replicas, MongoDB secondaries, Kafka consumers, CassandraDB (tunable consistency), DynamoDB global tables.

Code Example

```
class FollowerNode {
  private final LeaderNode leader;
  private long appliedIndex = 0;

  // Linearizable read: ask leader for its commit index, wait until we catch up
  public Object linearizableRead(String key) throws InterruptedException {
    long leaderCommit = leader.getCommitIndex();
    while (appliedIndex < leaderCommit) {
      Thread.sleep(1); // wait for replication to catch up
    }
    return localStore.get(key);
  }

  // Eventually-consistent read: just answer from local state
  public Object staleRead(String key) {
    return localStore.get(key); // may be slightly behind
  }
}
```

Majority Quorum

Consensus

PROBLEM	In a distributed system, any node can fail or be partitioned. How do you make decisions that remain valid even if some nodes are unavailable — without risking a split-brain?

SOLUTION	Require that any decision must be confirmed by more than half the nodes (majority = floor(n/2)+1). Because two majorities always overlap by at least one node, two conflicting decisions can never both be committed.

Advantages

- Mathematically guarantees no split-brain: two majorities share ≥1 node
- Tolerates (n-1)/2 node failures while remaining available
- Simple rule that underpins Paxos, Raft, Zab

USE THIS WHEN	Any time you need a system to remain consistent across node failures: leader election, committing log entries, distributed locks.

Code Example

```
class QuorumChecker {
  private final int totalNodes;

  public int majority() {
    return (totalNodes / 2) + 1;
  }

  // E.g., 5 nodes → majority = 3
  // Safe: can lose 2 nodes and still commit
  public boolean hasQuorum(Set<String> respondedNodes) {
    return respondedNodes.size() >= majority();
  }

  // Leader waits for quorum before committing
  public boolean tryCommit(Map<String,Boolean> acks) {
    long confirmed = acks.values().stream().filter(v -> v).count();
    return confirmed >= majority();
  }
}
```

Generation Clock

Consensus

PROBLEM	A node can be isolated (thinks it's leader), then rejoin and try to issue commands after a new leader was elected. How do nodes reject stale commands from deposed leaders?

SOLUTION	Each leader election increments a global generation (epoch/term) counter. All messages carry the sender's generation. Receivers ignore messages from lower generations than their own.

Advantages

- Eliminates split-brain without coordination overhead
- Cheap: a single integer per message
- Enables fast rejection of stale leaders

USE THIS WHEN	Raft (term), Paxos (ballot), ZooKeeper (epoch), Kafka (controller epoch). Required in any leader-follower system where leaders can be replaced.

Code Example

```
class GenerationClock {
  private volatile long generation = 0;

  // Called on every leader election
  public long incrementAndGet() {
    return ++generation;
  }

  // Accept message only if generation >= ours
  public boolean accept(long messageGeneration) {
    if (messageGeneration > generation) {
      generation = messageGeneration; // update to latest known
      return true;
    }
    return messageGeneration == generation;
  }
}

// In a node: attach generation to every outgoing RPC
AppendEntriesRequest req = new AppendEntriesRequest(
    clock.getGeneration(), entries);
// Receiver: if req.generation < myGeneration -> reject immediately
```

Paxos

Consensus

PROBLEM	Multiple proposers might try to commit different values concurrently. How do you reach a single, safe agreement even when messages are lost or nodes fail, using only two message round-trips?

SOLUTION	Two phases: (1) Prepare — a proposer asks all acceptors to promise not to accept any proposal with a lower number; (2) Accept — if a majority promise, the proposer sends the value for them to accept. A value is chosen when a majority accepts it.

Advantages

- Proven correct under asynchronous network + crash-stop failures
- Minimal assumptions: no synchrony required
- Foundation for Multi-Paxos (add a leader for efficiency)

USE THIS WHEN	Google Chubby, Apache Zookeeper's Zab, Google Spanner's TrueTime-based variant. Use when you need proven safety guarantees and are building consensus from scratch.

Code Example

```
// Single-decree Paxos (one consensus instance)
class Proposer {
  int proposalId = 0;
  Object value;

  // Phase 1: Prepare
  PrepareResult prepare(List<Acceptor> acceptors) {
    proposalId = maxSeenId + 1;
    int promises = 0; Object highestAccepted = null; int highestId = -1;
    for (Acceptor a : acceptors) {
      Promise p = a.prepare(proposalId);
      if (p.promised) {
        promises++;
        if (p.acceptedId > highestId) {
          highestId = p.acceptedId;
          highestAccepted = p.acceptedValue;
        }
      }
    }
    if (highestAccepted != null) value = highestAccepted; // must use this
    return new PrepareResult(promises >= majority(), value);
  }

  // Phase 2: Accept
  boolean accept(List<Acceptor> acceptors) {
    int acks = 0;
    for (Acceptor a : acceptors)
      if (a.accept(proposalId, value)) acks++;
    return acks >= majority();
  }
}
```

HeartBeat

Failure Detection

PROBLEM	In a distributed system, a node can fail silently. How do other nodes know when a peer has crashed vs. is just slow?

SOLUTION	Every node periodically sends a small heartbeat message to its peers (or a leader sends to followers). If a node does not receive a heartbeat within a timeout window, it is presumed dead.

Advantages

- Simple and universally applicable
- Low overhead: tiny messages at configurable intervals
- Enables automatic failover without human intervention

USE THIS WHEN	Every distributed system uses this: ZooKeeper session timeouts, Kafka broker liveness, Kubernetes pod health checks, TCP keepalive, Raft election timeouts.

Code Example

```
class HeartbeatManager {
  private final long timeoutMs;
  private final Map<String, Long> lastSeen = new ConcurrentHashMap<>();
  private final ScheduledExecutorService scheduler =
Executors.newScheduledThreadPool(1);

  public void start() {
    scheduler.scheduleAtFixedRate(this::sendHeartbeats, 0, 500,
TimeUnit.MILLISECONDS);
    scheduler.scheduleAtFixedRate(this::checkFailures, 0, 1000,
TimeUnit.MILLISECONDS);
  }

  public void onHeartbeatReceived(String nodeId) {
    lastSeen.put(nodeId, System.currentTimeMillis());
  }

  private void checkFailures() {
    long now = System.currentTimeMillis();
    lastSeen.forEach((nodeId, ts) -> {
      if (now - ts > timeoutMs) {
        onNodeFailed(nodeId);
      }
    });
  }
}
```

Single-Socket Channel

Communication

PROBLEM	Nodes communicating over multiple TCP connections can receive replies out of order. How do you preserve message ordering between two nodes?

SOLUTION	Maintain exactly one TCP connection between any two nodes. Since TCP guarantees in-order delivery within a single connection, message ordering is preserved. All requests between the pair are serialized over this channel.

Advantages

- Message ordering guaranteed at no algorithmic cost
- Reduces connection overhead vs. per-request connections
- Simplifies reasoning about message sequencing

USE THIS WHEN	ZooKeeper client-server communication, Raft inter-node RPCs, Kafka broker connections. Any system where ordering guarantees between peers matter.

Code Example

```
class SingleSocketChannel {
  private final Socket socket;
  private final ObjectOutputStream out;
  private final ObjectInputStream in;
  private final Object writeLock = new Object();

  // One connection per peer, reused for all messages
  public SingleSocketChannel(String host, int port) throws IOException {
    this.socket = new Socket(host, port);
    this.out = new ObjectOutputStream(socket.getOutputStream());
    this.in  = new ObjectInputStream(socket.getInputStream());
  }

  public void send(Message msg) throws IOException {
    synchronized (writeLock) { // serialize writes
      out.writeObject(msg);
      out.flush();
    }
  }
}
```

Request Pipeline

Communication

PROBLEM	Sending one request, waiting for its response, then sending the next (stop-and-wait) wastes network bandwidth and increases latency unnecessarily.

SOLUTION	Send multiple requests over the single-socket channel without waiting for each reply. Responses are matched to requests by correlation ID when they arrive.

Advantages

- Dramatically improves throughput by keeping the network pipe full
- Reduces effective latency for batch operations
- Used by Kafka producers/consumers for high-throughput I/O

USE THIS WHEN	High-throughput log append scenarios, Kafka producer acks, Redis pipelining, any network-bound replication path.

Code Example

```
class RequestPipeline {
  private final SingleSocketChannel channel;
  // Pending requests waiting for response
  private final Map<Long, CompletableFuture<Response>> pending = new
ConcurrentHashMap<>();
  private final AtomicLong correlationId = new AtomicLong();

  // Fire-and-don't-wait — returns future immediately
  public CompletableFuture<Response> send(Request req) throws IOException {
    long id = correlationId.incrementAndGet();
    CompletableFuture<Response> future = new CompletableFuture<>();
    pending.put(id, future);
    channel.send(new Message(id, req));
    return future; // caller awaits when ready
  }

  // Background thread matches responses to futures
  private void onResponse(Response resp) {
    CompletableFuture<Response> f = pending.remove(resp.getCorrelationId());
    if (f != null) f.complete(resp);
  }
}
```

Request Batch

Communication

PROBLEM	Sending thousands of small messages individually saturates the network and creates per-message overhead in serialization and I/O syscalls.

SOLUTION	Buffer multiple requests and send them together as a single network payload. The receiver unpacks and processes them in bulk.

Advantages

- Reduces number of syscalls and network round-trips
- Better compression ratios (similar data compresses together)
- Kafka's micro-batching achieves multi-GB/s throughput this way

USE THIS WHEN	Kafka producer batching, Cassandra batch writes, gRPC streaming, database bulk inserts, any high-throughput producer-consumer system.

Code Example

```
class BatchSender {
  private final List<Request> buffer = new ArrayList<>();
  private final int maxBatchSize;
  private final long maxLingerMs;

  public synchronized void add(Request req) throws IOException {
    buffer.add(req);
    if (buffer.size() >= maxBatchSize) flush();
  }

  public synchronized void flush() throws IOException {
    if (buffer.isEmpty()) return;
    List<Request> batch = new ArrayList<>(buffer);
    buffer.clear();
    channel.send(new BatchRequest(batch)); // single network write
  }

  // Timer-based flush: don't wait forever if batch is small
  @Scheduled(fixedDelay = 5) // maxLingerMs
  public void timerFlush() throws IOException { flush(); }
}
```

Request Waiting List

Communication

PROBLEM	A client sends a request that requires confirmation from multiple cluster nodes (e.g., a quorum write). The handling node needs to suspend the client request until the quorum responds.

SOLUTION	Maintain a waiting list of pending client requests, keyed by the log index or request ID. When enough peer acknowledgments arrive, complete the waiting request and respond to the client.

Advantages

- Decouples request handling from quorum collection
- Avoids blocking threads while waiting for peer responses
- Clean integration with async I/O frameworks

USE THIS WHEN	Raft leaders waiting for follower acks before responding to clients, ZooKeeper quorum writes, any leader that must collect peer votes before replying.

Code Example

```
class RequestWaitingList {
  // Map from log index to pending client response
  private final Map<Long, CompletableFuture<Object>> waitingRequests = new
ConcurrentHashMap<>();
  private final Map<Long, Set<String>> ackSets = new ConcurrentHashMap<>();
  private final int majority;

  public CompletableFuture<Object> register(long logIndex) {
    CompletableFuture<Object> future = new CompletableFuture<>();
    waitingRequests.put(logIndex, future);
    ackSets.put(logIndex, ConcurrentHashMap.newKeySet());
    return future;
  }

  public void onAck(String nodeId, long logIndex) {
    Set<String> acks = ackSets.get(logIndex);
    if (acks == null) return;
    acks.add(nodeId);
    if (acks.size() >= majority) {
      CompletableFuture<Object> f = waitingRequests.remove(logIndex);
      if (f != null) f.complete(logIndex);
    }
  }
}
```

Singular Update Queue

Communication

PROBLEM	Multiple concurrent threads updating shared state require careful locking. How do you handle updates safely without complex synchronization while keeping the caller non-blocking?

SOLUTION	Funnel all updates through a single-threaded executor backed by an unbounded queue. Callers submit work items and get a Future back. The single worker thread processes items sequentially — no locks needed.

Advantages

- Eliminates data races on shared state without locks
- Callers remain non-blocking (fire and future)
- Natural back-pressure: queue size is observable

USE THIS WHEN	ZooKeeper's request processor pipeline, Kafka's log append path, actor frameworks (Akka), any component whose internal state must be updated serially.

Code Example

```
class SingularUpdateQueue<T, R> {
  private final ExecutorService executor = Executors.newSingleThreadExecutor();
  private final Function<T, R> handler;

  public SingularUpdateQueue(Function<T, R> handler) {
    this.handler = handler;
  }

  // Non-blocking: returns a future, processed in order by single thread
  public CompletableFuture<R> submit(T request) {
    return CompletableFuture.supplyAsync(
        () -> handler.apply(request), executor
    );
  }
}

// Usage: all writes go through one queue — no synchronization needed
SingularUpdateQueue<WriteRequest, WriteResponse> writeQueue =
    new SingularUpdateQueue<>(this::applyWrite);
```

Fixed Partitions

Partitioning

PROBLEM	You need to distribute data across multiple nodes (sharding). If you assign data to nodes directly, adding or removing nodes requires moving huge amounts of data.

SOLUTION	Create a fixed, large number of virtual partitions (e.g., 1024). Assign partitions to nodes. When you add/remove a node, only reassign partitions — not individual data items. Data-to-partition mapping never changes.

Advantages

- Adding/removing nodes only moves partitions, not raw data
- Partition count is predictable — no surprises at scale
- Each partition can be replicated independently

USE THIS WHEN	Kafka (fixed partition count per topic), Redis Cluster (16,384 hash slots), Cassandra (virtual nodes). Use when you need a stable data distribution scheme.

Code Example

```
class FixedPartitionRouter {
  private final int numPartitions;
  private final Map<Integer, String> partitionToNode; // partition -> node address
```

```
  // Assign a key to a partition — this mapping NEVER changes
  public int getPartition(String key) {
    return Math.abs(key.hashCode()) % numPartitions;
  }

  public String getNode(String key) {
    return partitionToNode.get(getPartition(key));
  }

  // On node failure: just update partitionToNode mapping
  // Data-to-partition mapping is unchanged
  public void reassignPartition(int partitionId, String newNode) {
    partitionToNode.put(partitionId, newNode);
  }
}
```

Key-Range Partitions

Partitioning

PROBLEM	Hash-based partitioning scatters data randomly. Range queries ("give me all users with IDs 1000–2000") must hit every partition — very inefficient.

SOLUTION	Sort data by key and assign contiguous key ranges to partitions. Partition 1 owns keys A–M, partition 2 owns N–Z, etc. Range queries can be routed to only the relevant partitions.

Advantages

- Range queries are efficient: hit only the relevant partitions
- Sorted scans are possible within a partition
- Natural for time-series data (partition by date range)

USE THIS WHEN	HBase, Cassandra (compound key ranges), CockroachDB, BigTable, DynamoDB table scans, time-series databases. Use when your access patterns are range-heavy.

Code Example

```
class KeyRangePartitioner {
  // Sorted list of partition boundaries
  private final TreeMap<String, String> rangeBoundaries = new TreeMap<>();
  // Maps partition start key -> node address

  public KeyRangePartitioner() {
```

```
    // Partition 0: keys < 'g', Partition 1: 'g' <= keys < 'n', etc.
    rangeBoundaries.put("", "node1"); // start of keyspace
    rangeBoundaries.put("g", "node2");
    rangeBoundaries.put("n", "node3");
  }

  public String getNode(String key) {
    // Find the partition whose range includes this key
    return rangeBoundaries.floorEntry(key).getValue();
  }

  // Range query: ['alice', 'bob'] -> only hits node1
  public List<String> getNodesForRange(String start, String end) {
    return new ArrayList<>(rangeBoundaries.subMap(start, true, end, true).values());
  }
}
```

Two-Phase Commit (2PC)

Partitioning

PROBLEM	A transaction spans multiple nodes (e.g., debit account on node A, credit on node B). How do you ensure either both happen or neither — atomically, even if a node fails mid-way?

SOLUTION	Phase 1 (Prepare): coordinator asks all participants to tentatively reserve resources and vote Yes/No. Phase 2 (Commit/Abort): if all vote Yes, coordinator tells everyone to commit. If any vote No or timeout, coordinator tells everyone to abort.

Advantages

- Atomic cross-node transactions — the distributed ACID guarantee
- Participants independently verify they can commit before voting
- Well-understood protocol with clear failure modes

USE THIS WHEN	Distributed databases (MySQL NDB, CockroachDB), XA transactions, any operation that must atomically affect data on multiple independent nodes.

Code Example

```
class TwoPhaseCommitCoordinator {
  enum TxState { PREPARING, COMMITTED, ABORTED }

  public boolean execute(List<Participant> participants, Transaction tx) {
    String txId = UUID.randomUUID().toString();
```

```
    // Phase 1: Prepare — can everyone commit?
    boolean allReady = participants.stream()
        .allMatch(p -> p.prepare(txId, tx));

    // Phase 2: Commit or Abort
    if (allReady) {
      participants.forEach(p -> p.commit(txId));
      return true;
    } else {
      participants.forEach(p -> p.abort(txId));
      return false;
    }
    // Note: coordinator crash between phases requires a WAL to recover
  }
}
```

Versioned Value

Distributed Time

PROBLEM	Multiple clients update the same key. How do you keep history, detect stale reads, and implement multi-version concurrency control (MVCC)?

SOLUTION	Store each value along with a version number. When a key is updated, a new version is created rather than overwriting. Readers can request a specific version or the latest. Old versions are garbage-collected eventually.

Advantages

- Historical reads: time-travel queries at any past version
- MVCC: readers don't block writers, writers don't block readers
- Conflict detection: compare versions to detect stale writes

USE THIS WHEN	etcd (MVCC store), CockroachDB (MVCC), Cassandra (timestamps), git (commit hashes). Any system needing optimistic concurrency control or historical reads.

Code Example

```
class VersionedStore {
  // key -> sorted list of (version, value) pairs
  private final Map<String, TreeMap<Long, byte[]>> store = new ConcurrentHashMap<>();
  private final AtomicLong versionClock = new AtomicLong();

  public long put(String key, byte[] value) {
    long version = versionClock.incrementAndGet();
```

```
    store.computeIfAbsent(key, k -> new TreeMap<>()).put(version, value);
    return version;
  }

  public byte[] get(String key) { // latest
    TreeMap<Long, byte[]> versions = store.get(key);
    return versions == null ? null : versions.lastEntry().getValue();
  }

  // Read as-of a specific version (time-travel query)
  public byte[] getAtVersion(String key, long version) {
    TreeMap<Long, byte[]> versions = store.get(key);
    if (versions == null) return null;
    Map.Entry<Long, byte[]> e = versions.floorEntry(version);
    return e == null ? null : e.getValue();
  }
}
```

Version Vector

Distributed Time

PROBLEM	In a system with multiple replicas that accept concurrent writes, how do you detect when two updates conflict vs. when one is a direct successor of the other?

SOLUTION	Maintain a vector of per-node counters. Each node increments its own counter on every write and includes the full vector in the value. On merge: if vector A dominates vector B (every component >=), A is newer. If neither dominates, they conflict.

Advantages

- Distinguishes causal succession from true conflicts
- No central coordinator required
- Amazon Dynamo used this for eventual consistency

USE THIS WHEN	Multi-master replication (Amazon Dynamo, Riak, CouchDB), shopping cart merges, any leaderless eventual-consistency system.

Code Example

```
class VersionVector {
  private final Map<String, Long> counters = new HashMap<>();

  public void increment(String nodeId) {
    counters.merge(nodeId, 1L, Long::sum);
```

```
  }

  // Does 'this' dominate 'other'? (this is causally after or equal to other)
  public boolean dominates(VersionVector other) {
    for (Map.Entry<String,Long> e : other.counters.entrySet()) {
      if (counters.getOrDefault(e.getKey(), 0L) < e.getValue())
        return false; // we're behind on this node
    }
    return true;
  }

  // true = conflict (neither dominates the other)
  public boolean conflictsWith(VersionVector other) {
    return !this.dominates(other) && !other.dominates(this);
  }

  // Merge: take max of each component
  public VersionVector merge(VersionVector other) {
    VersionVector merged = new VersionVector();
    Set<String> allKeys = new HashSet<>(counters.keySet());
    allKeys.addAll(other.counters.keySet());
    for (String k : allKeys)
      merged.counters.put(k, Math.max(
          counters.getOrDefault(k,0L),
          other.counters.getOrDefault(k,0L)));
    return merged;
  }
}
```

Lamport Clock

Distributed Time

PROBLEM	Wall-clock timestamps on different servers are never perfectly synchronized. You cannot use them to determine the order of events across nodes.

SOLUTION	Each node maintains a logical counter. On any event, increment it. When sending a message, include the counter. When receiving, take max(local, received) + 1. This gives a partial ordering consistent with causality.

Advantages

- Total ordering of all events across nodes
- No hardware clock required — purely logical
- Cheap: a single integer per message

USE THIS WHEN	Distributed databases needing causal ordering, distributed debugging/tracing, event sourcing across services. Building block for Hybrid Clock.

Code Example

```
class LamportClock {
  private long time = 0;

  // On any local event (write, read, etc.)
  public synchronized long tick() {
    return ++time;
  }

  // On sending a message: attach current time
  public synchronized long getTime() { return time; }

  // On receiving a message with sender's timestamp
  public synchronized long update(long receivedTime) {
    time = Math.max(time, receivedTime) + 1;
    return time;
  }
}
```

Hybrid Clock

Distributed Time

PROBLEM	Lamport Clocks give ordering but not human-readable timestamps. Wall clocks give timestamps but can go backwards (NTP). Can you have both?

SOLUTION	Hybrid Logical Clock (HLC) = max(wallClock, lastKnownTime) + logical counter. Time always moves forward. Timestamps are close to real time. Causality is preserved.

Advantages

- Human-readable timestamps close to real wall time
- Monotonically increasing — never goes backwards
- Causality tracking like Lamport Clock

USE THIS WHEN	CockroachDB, YugabyteDB, MongoDB oplog timestamps. Any globally distributed system that wants real-time proximity AND causal ordering.

Code Example

```
class HybridClock {
  private long wallTime;  // physical ms
```

```
  private long logical;   // tie-breaker counter

  public synchronized HLCTimestamp now() {
    long pt = System.currentTimeMillis();
    if (pt > wallTime) {
      wallTime = pt; logical = 0;
    } else {
      logical++; // wall clock didn't advance — use logical
    }
    return new HLCTimestamp(wallTime, logical);
  }

  // On receiving a message timestamp
  public synchronized HLCTimestamp update(HLCTimestamp received) {
    long pt = System.currentTimeMillis();
    long maxWall = Math.max(Math.max(wallTime, received.wallTime), pt);
    if (maxWall == wallTime && maxWall == received.wallTime)
      logical = Math.max(logical, received.logical) + 1;
    else if (maxWall == wallTime)
      logical++;
    else if (maxWall == received.wallTime)
      logical = received.logical + 1;
    else
      logical = 0;
    wallTime = maxWall;
    return new HLCTimestamp(wallTime, logical);
  }
}
```

Clock-Bound Wait

Distributed Time

PROBLEM	Even with Hybrid Clocks, different nodes have slightly different wall clocks. A write on node A might get a timestamp that appears before a concurrent write on node B, breaking causality.

SOLUTION	Before completing a write, wait for the clock uncertainty window to pass (e.g., wait ε milliseconds where ε is the maximum clock skew). After waiting, any subsequent event on any node is guaranteed to have a higher timestamp.

Advantages

- Enables external consistency (linearizability) without a central clock
- Allows safe cross-node timestamp ordering
- Google Spanner TrueTime uses this for globally serializable transactions

USE THIS WHEN	Google Spanner, CockroachDB's commit-wait, any globally distributed database that promises linearizable transactions across data centers.

Code Example

```
class ClockBoundWait {
  private final TrueTimeOracle truetime; // returns (earliest, latest) for 'now'

  // Guarantee: after this method returns, s < now() on ALL nodes
  public void waitUntilAfter(long commitTimestamp) throws InterruptedException {
    // Keep polling until our earliest possible 'now' > commitTimestamp
    while (truetime.now().earliest <= commitTimestamp) {
      Thread.sleep(1); // typically only 1-7ms in Google's network
    }
    // Now safe: any future event on any node is strictly after commitTimestamp
  }

  // In practice for a transaction commit:
  public void commitTransaction(Transaction tx) throws InterruptedException {
    long ts = truetime.now().latest; // worst-case 'now'
    tx.setCommitTimestamp(ts);
    tx.writeToLog();
    waitUntilAfter(ts); // commit-wait
    tx.markVisible();  // now linearizable
  }
}
```

Consistent Core

Cluster Management

PROBLEM	Building quorum-based consensus into a large data cluster (thousands of nodes) is expensive. Every metadata operation would require communicating with a huge majority.

SOLUTION	Use a small, separate, strongly-consistent cluster (e.g., 3 or 5 nodes running Raft/Paxos) to handle coordination: leader election, cluster membership, distributed locks. The large data cluster simply uses this as an oracle.

Advantages

- Large data cluster avoids expensive quorum protocols
- Coordination is handled by proven, purpose-built systems
- Clean separation: data plane vs. control plane

USE THIS WHEN	Kafka + ZooKeeper/KRaft, HDFS + ZooKeeper, Kubernetes + etcd, Elasticsearch + ZooKeeper. Use whenever your data cluster needs coordination but is too large for quorum overhead.

Code Example

```
// Data cluster node uses Consistent Core (e.g. ZooKeeper/etcd) for coordination
class DataNode {
  private final ConsistentCore core; // e.g., ZooKeeper client
  private final String nodeId;

  // Leader election via consistent core
  public void participateInElection() {
    core.createEphemeralSequential("/election/candidate-", nodeId);
    String leader = core.getChildren("/election")
        .stream().sorted().findFirst().orElseThrow();
    if (leader.contains(nodeId)) becomeLeader();
    else watchForLeaderFailure(leader);
  }

  // Distributed lock for critical section
  public boolean acquireLock(String resource) {
    return core.create("/locks/" + resource,
        nodeId, CreateMode.EPHEMERAL); // auto-released on disconnect
  }
}
```

Lease

Cluster Management

PROBLEM	A node holds a lock or leadership role. If it crashes, other nodes must wait for a timeout before assuming it's dead. How do you bound that wait time reliably?

SOLUTION	Grant time-bounded leases. A node holding a lease is allowed to act as leader/lock-holder only until the lease expires. It must renew before expiry. If it crashes, others can safely take over after the lease period.

Advantages

- Bounded recovery time — no indefinite wait
- Prevents two nodes acting as leader simultaneously
- Simpler than explicit lock release (lease just expires)

USE THIS WHEN	Kubernetes leader election, GFS master leases, distributed locks (etcd lease TTL), HBase region server assignment, any system needing fencing tokens.

Code Example

```
class LeaseManager {
```

```
  private volatile long leaseExpiryMs = 0;
  private final long leaseDurationMs;
  private final ConsistentCore core;
  private final String nodeId;

  public boolean acquireOrRenewLease(String resource) {
    // Try to set lease in consistent core with TTL
    boolean acquired = core.putIfAbsentWithTTL(
        "/leases/" + resource, nodeId, leaseDurationMs);
    if (acquired) {
      leaseExpiryMs = System.currentTimeMillis() + leaseDurationMs;
      scheduleRenewal(resource);
    }
    return acquired;
  }

  // Node MUST check before every privileged action
  public boolean isLeaseValid() {
    return System.currentTimeMillis() < leaseExpiryMs;
  }
}
```

State Watch

Cluster Management

PROBLEM	Clients need to react to changes in cluster state (e.g., new leader elected, config changed). Polling the consistent core wastes resources and adds latency.

SOLUTION	Let clients register watchers on specific keys or paths. The server notifies all registered watchers immediately when the watched value changes.

Advantages

- Push model: no polling overhead, instant notification
- Enables reactive cluster reconfiguration
- Foundation for service discovery and distributed coordination

USE THIS WHEN	ZooKeeper watches (the foundational use case), etcd watch API, Consul health check callbacks, Kubernetes informer/watch pattern.

Code Example

```
class StateWatchService {
  private final Map<String, List<Consumer<WatchEvent>>> watchers = new
ConcurrentHashMap<>();
```

```
  // Client registers interest in a key
  public void watch(String key, Consumer<WatchEvent> callback) {
    watchers.computeIfAbsent(key, k -> new CopyOnWriteArrayList<>()).add(callback);
  }

  // Called internally when a key's value changes
  public void notifyWatchers(String key, Object newValue, EventType type) {
    List<Consumer<WatchEvent>> callbacks = watchers.remove(key); // one-time watch
    if (callbacks != null) {
      WatchEvent event = new WatchEvent(key, newValue, type);
      callbacks.forEach(cb -> cb.accept(event));
    }
  }
}

// Usage: watch for leader changes
core.watch("/election/leader", event -> {
  if (event.getType() == DELETED) rerunElection();
});
```

Gossip Dissemination

Cluster Management

PROBLEM	In a large cluster, how do you propagate information (node failures, membership changes, configuration updates) to all nodes without a central coordinator and without flooding the network?

SOLUTION	Each node periodically picks a random subset of peers and shares its current knowledge. Recipients merge the new information with their own and propagate in the next cycle. Like a rumour, information reaches the whole cluster in O(log N) cycles.

Advantages

- Decentralized — no single point of failure
- Scales to thousands of nodes without flood overhead
- Eventually consistent by design — perfect for non-critical metadata

USE THIS WHEN	Cassandra (failure detection, ring membership), DynamoDB, Riak, Consul, SWIM protocol. Any large cluster needing decentralized membership/health propagation.

Code Example

```
class GossipNode {
  private final Map<String, NodeState> clusterState = new ConcurrentHashMap<>();
  private final List<String> knownPeers;
```

```
  private final int fanout = 3; // nodes to gossip with each cycle

  // Every gossip interval (e.g., 1 second)
  public void gossip() throws IOException {
    // Pick 'fanout' random peers
    Collections.shuffle(knownPeers);
    List<String> targets = knownPeers.subList(0, Math.min(fanout, knownPeers.size()));
    for (String peer : targets) {
      sendGossip(peer, clusterState); // send full or delta state
    }
  }

  // On receiving gossip from a peer
  public void onGossipReceived(Map<String, NodeState> peerState) {
    peerState.forEach((nodeId, state) -> {
      NodeState current = clusterState.get(nodeId);
      // Keep the state with the higher version (heartbeat counter)
      if (current == null || state.getVersion() > current.getVersion()) {
        clusterState.put(nodeId, state);
      }
    });
  }
}
```

Emergent Leader

Cluster Management

PROBLEM	Running an explicit election protocol on every node startup/failure adds complexity and latency. Is there a simpler way to select a leader without votes?

SOLUTION	Order nodes by their age in the cluster (join time). The oldest surviving node is always the leader — no election needed. If it leaves, the next oldest takes over automatically.

Advantages

- No explicit election protocol — zero coordination overhead
- Deterministic: no ties, no split-brain possible
- Simple to implement with a Consistent Core tracking join order

USE THIS WHEN	Kafka KRaft controller (oldest active broker wins), ZooKeeper leader selection among candidate nodes, Kubernetes lease-based leader election fallback.

Code Example

```
class EmergentLeaderSelector {
```

```
  private final ConsistentCore core;
  private final String myNodeId;
  private final long myJoinTime;

  public void register() {
    // Register with join timestamp in consistent core
    core.put("/members/" + myNodeId,
        String.valueOf(myJoinTime), CreateMode.EPHEMERAL);
  }

  public boolean amILeader() {
    // Leader = member with the smallest (earliest) join timestamp
    return core.getChildren("/members")
        .stream()
        .map(id -> Map.entry(id, Long.parseLong(core.get("/members/" + id))))
        .min(Map.Entry.comparingByValue())
        .map(e -> e.getKey().equals(myNodeId))
        .orElse(false);
  }
}
```

Idempotent Receiver

Client Patterns

PROBLEM	Network failures cause clients to retry requests. If the server already processed the first attempt, a retry causes a duplicate operation (e.g., a payment processed twice).

SOLUTION	Clients assign a unique request ID to each logical operation. The server stores a cache of recently processed request IDs. Before processing, check: if already processed, return the cached result. Otherwise process and cache.

Advantages

- Retries are safe — duplicate requests are silently deduplicated
- At-least-once delivery becomes effectively exactly-once
- Simple: just a map of {requestId -> response}

USE THIS WHEN	Payment APIs, Kafka producer idempotence (producer ID + sequence number), database upserts with unique keys, any at-least-once messaging system that needs exactly-once semantics.

Code Example

```
class IdempotentServer {
  // Cache: requestId -> already-computed response
```

```
  private final Map<String, Response> processedRequests = new ConcurrentHashMap<>();
  // Optional: evict old entries after TTL (Guava Cache works well here)

  public Response handle(Request req) {
    String requestId = req.getClientId() + ":" + req.getSequenceNumber();

    // Already processed? Return cached result
    Response cached = processedRequests.get(requestId);
    if (cached != null) return cached;

    // First time: process and cache
    Response response = processRequest(req);
    processedRequests.put(requestId, response);
    return response;
  }
}

// Client assigns IDs and retries safely
class IdempotentClient {
  private final String clientId = UUID.randomUUID().toString();
  private final AtomicLong seq = new AtomicLong();

  public Response sendWithRetry(Object payload) {
    Request req = new Request(clientId, seq.incrementAndGet(), payload);
    while (true) {
      try { return server.handle(req); }
      catch (NetworkException e) { /* retry */ }
    }
  }
}
```

Choosing Patterns by Scenario

Building a Replicated State Machine (e.g., etcd, ZooKeeper clone)

Write-Ahead Log → Replicated Log → Leader and Followers → Majority Quorum → Generation Clock → High-Water Mark → HeartBeat → Singular Update Queue → Request Waiting List → Idempotent Receiver

Building a Distributed Message Log (e.g., Kafka clone)

Write-Ahead Log → Segmented Log → Low-Water Mark → Leader and Followers → High-Water Mark → Follower Reads → Fixed Partitions → Request Pipeline → Request Batch

Building a Distributed Database (e.g., CockroachDB clone)

Write-Ahead Log → Versioned Value → Two-Phase Commit → Key-Range Partitions → Hybrid Clock → Clock-Bound Wait → Version Vector → Replicated Log → Idempotent Receiver

Building Cluster Coordination (e.g., the coordination layer itself)

Consistent Core → Lease → State Watch → Gossip Dissemination → Emergent Leader → HeartBeat → Generation Clock → Single-Socket Channel

Handling Time Across Distributed Nodes

Lamport Clock → Hybrid Clock → Clock-Bound Wait → Versioned Value → Version Vector

Scaling Reads Without Sacrificing Consistency

Follower Reads → Versioned Value → High-Water Mark → Consistent Core

How the Patterns Interrelate

No pattern lives in isolation. Here are the most important chains:

> **Write-Ahead Log** *is split into segments by* **Segmented Log** *which is cleaned up by* **Low-Water Mark**
>
> **Leader and Followers** *decides commits via* **Majority Quorum** *marks them with* **High-Water Mark** *defended against stale leaders by* **Generation Clock**
>
> **Replicated Log** *is built from* **Write-Ahead Log** *replicated via* **Leader and Followers** *with ordering from* **Request Waiting List**
>
> **Consistent Core** *gives leases via* **Lease** *with change notification via* **State Watch** *failure detection via* **HeartBeat**
>
> **Two-Phase Commit** *tracks versions with* **Versioned Value** *orders them with* **Hybrid Clock** *and waits with* **Clock-Bound Wait**